

UNIDAD 2 — CLASES, OBJETOS Y TIPOS DE DATOS

Autor: Ing. Gaston Genaro Quelali Calcina

Contenido:

- [1.1 Definición y estructura de clases](#)
 - [1.2 Objetos, instanciación y alcance de variables](#)
 - [1.3 Modificadores de acceso, constructores y destructores](#)
 - [1.4 Sobrecarga y sobreescritura de métodos](#)
 - [1.5 Librerías estándar y externas \(gestión de dependencias\)](#)
 - [Preguntas de repaso](#)
 - [Glosario](#)
-

1.1 Definición y estructura de clases

1.1.1 ¿Qué es una clase?

Una **clase** es la **plantilla** que define: - **Atributos** (estado): los datos que guarda cada objeto. - **Métodos** (comportamiento): las operaciones que puede realizar.



Anatomía de una clase en UML

Figura: Anatomía de una clase en UML

Símbolo	Significado
-	Privado (solo accesible dentro de la clase)
+	Público (accesible desde cualquier lugar)
#	Protegido (accesible para subclasses)

1.1.2 Anatomía en código Java

```
public class CuentaBancaria {  
  
    // Atributos (estado del objeto)  
    private double saldo;  
    private String titular;  
    private String numero;  
  
    // Constructor (crea e inicializa el objeto)  
    public CuentaBancaria(String titular, String numero) {  
        this.titular = titular;  
        this.numero = numero;  
        this.saldo = 0.0;  
    }  
}
```

```
// Métodos (comportamiento)
public void depositar(double monto) {
    if (monto > 0) {
        this.saldo += monto;
    }
}

public boolean retirar(double monto) {
    if (monto > 0 && monto <= this.saldo) {
        this.saldo -= monto;
        return true;
    }
    return false;
}

public double consultarSaldo() {
    return this.saldo;
}
}
```

1.2 Objetos, instanciación y alcance de variables

1.2.1 Instanciación

Un **objeto** es una **instancia** de una clase. Se crea con el operador `new` :

```
CuentaBancaria cuenta = new CuentaBancaria("Ana Pérez", "001-2345");
```

1. `new` reserva memoria para el objeto.
2. El **constructor** inicializa los atributos.
3. La variable `cuenta` guarda una **referencia** al objeto.

1.2.2 Ciclo de vida de un objeto



Figura: Modificadores de acceso en Java

Dato Java: el **Garbage Collector (GC)** libera automáticamente la memoria de los objetos que ya no tienen referencias. No hay `delete` como en C++.

1.2.3 Alcance de variables

Tipo de variable	Dónde vive	Alcance
Local	Dentro de un método	Solo dentro de ese método
De instancia	En la clase (sin <code>static</code>)	Cada objeto tiene su propia copia
De clase	Con <code>static</code>	Compartida por todos los objetos

```
public class Contador {
    private int instancia = 0;    // de instancia: cada objeto tiene la suya
    private static int total = 0; // de clase: compartida

    public void incrementar() {
        int local = 5;           // local: solo vive en este método
        this.instancia += local;
        Contador.total += 1;
    }
}
```

1.3 Modificadores de acceso, constructores y destructores

1.3.1 Modificadores de acceso

 Ciclo de vida de un objeto

Figura: Ciclo de vida de un objeto

Modificador	Misma clase	Mismo paquete	Subclase	Cualquier clase
<code>private</code>	✓	✗	✗	✗
(ninguno)	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

Regla práctica: atributos `private` , métodos de servicio `public` , y `protected` para lo que debe heredarse.

1.3.2 Constructores

- Tienen **el mismo nombre** de la clase y **no devuelven** tipo.
- Inician los atributos al crear el objeto.
- Si no definimos ninguno, Java crea el **constructor por defecto** (sin parámetros).

```
public class Libro {
    private String titulo;

    // Constructor explícito
    public Libro(String titulo) {
```

```

        this.titulo = titulo;
    }

    // Sobrecarga de constructor: sin parámetro
    public Libro() {
        this.titulo = "Sin título";
    }
}

```

1.3.3 Destruyores

En Java **no existen destruyores**: el **Garbage Collector** se encarga de liberar memoria. La alternativa para "limpiar" recursos (archivos, conexiones) es:

```

// Patrón try-with-resources (Java 7+)
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    // se cierra automáticamente al terminar
}

```

La unidad 4 (E/S) profundiza en esto. Por ahora: **recuerda que en Java el GC reemplaza a los destruyores.**

1.4 Sobrecarga y sobreescritura de métodos

1.4.1 Sobrecarga (overloading)

Mismo nombre, **distinta lista de parámetros** (cantidad o tipos), en la misma clase:

```

public class Calculadora {
    public int sumar(int a, int b) {
        return a + b;
    }

    public double sumar(double a, double b) {
        return a + b;
    }

    public int sumar(int a, int b, int c) {
        return a + b + c;
    }
}

```

El compilador elige el método correcto según los **argumentos** pasados.

1.4.2 Sobreescritura (overriding)

La clase hija **redefine** un método heredado con la misma firma:

```
public class Vehiculo {
    public void mover() {
        System.out.println("El vehículo se mueve");
    }
}

public class Auto extends Vehiculo {
    @Override
    public void mover() {
        System.out.println("El auto avanza por la ruta");
    }
}
```

Aspecto	Sobrecarga	Sobreescritura
Clases	Misma clase	Clase padre → hija
Firma	Cambian los parámetros	Misma firma
Motivo	Flexibilidad	Especializar el comportamiento

La sobreescritura es la base del **polimorfismo** que profundizaremos en la Unidad 4.

1.5 Librerías estándar y externas (gestión de dependencias)

1.5.1 Librería estándar vs externa

- **Librería estándar (JDK):** clases que vienen con Java (`java.util` , `java.io` , `java.time` ...). No requieren instalación.
- **Librería externa:** desarrolladas por terceros (Jackson para JSON, JUnit para pruebas, Log4j para logs). Se integran como **dependencias**.

1.5.2 Maven y Gradle

 Gestión de dependencias con Maven y Gradle

Figura: Gestión de dependencias con Maven y Gradle

Ejemplo de `pom.xml` (Maven):

```
<dependencies>
  <!-- Jackson: procesamiento de JSON -->
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
```

```
</dependency>
</dependencies>
```

Comandos básicos de Maven:

Comando	Qué hace
<code>mvn compile</code>	Compila el proyecto
<code>mvn test</code>	Ejecuta las pruebas
<code>mvn package</code>	Genera el <code>.jar</code> ejecutable
<code>mvn clean</code>	Elimina archivos compilados

Beneficio: declaras la dependencia **una vez** y Maven la descarga (con sus versiones correctas) desde el repositorio central. Adiós a "bajar el jar a mano".

Preguntas de repaso

1. ¿Cuáles son las partes de una clase? Da un ejemplo propio.
2. ¿Qué hace el operador `new` en Java?
3. Explica la diferencia entre variable **local**, **de instancia** y **de clase**.
4. ¿Qué modificadores de acceso existen y cuándo usarías cada uno?
5. ¿Para qué sirve el constructor? ¿Qué es el constructor por defecto?
6. ¿Por qué Java no necesita destructores? ¿Qué lo reemplaza?
7. Diferencia **sobrecarga** y **sobreescritura** con un ejemplo.
8. ¿Qué problema resuelve Maven en la gestión de dependencias?
9. Escribe una clase `Producto` con atributos `nombre`, `precio` y `stock` (private), constructor y métodos `getPrecio()` y `aplicarDescuento(double)`.

Glosario

Término	Definición
Clase	Plantilla con atributos y métodos
Objeto / Instancia	Creación concreta de una clase mediante <code>new</code>
Constructor	Método especial que inicializa el objeto al crearlo
Garbage Collector	Mecanismo de Java que libera memoria automáticamente
Sobrecarga	Mismo nombre de método, distintos parámetros
Sobreescritura	Redefinir un método heredado en la clase hija
Dependencia	Librería externa que el proyecto necesita
Maven / Gradle	Herramientas de gestión de dependencias y construcción